

abr. 27, 2026

Integrações Assíncronas - Mensageria

convidado Bruno Joaquim Cassia Dobler Fabio Diniz Francisco Schneider
Gabriel Severo Gabriel kohlrausch Ruan Rodrigues
aline.bianchini@t-systems.com ana.santos@t-systems.com
analista.kuehn@outlook.com anderson.hilario@t-systems.com
anderson.perim@t-systems.com andersongabriel_maria@hotmail.com
anselmogabriel421@gmail.com bmf.pedro@gmail.com
bruna.nery@t-systems.com bruno.destro@t-systems.com
caique.rodrigues@t-systems.com d.silva@t-systems.com
danilosilva2017@hotmail.com desenvolvedor@tidsoftware.com.br
douglas.nishiyama@t-systems.com dpsnbsb79@gmail.com
eduardo.kubota@t-systems.com eudy.amparo@t-systems.com
fabio.marinelli@t-systems.com farley.t.i@hotmail.com
felipeneves089@gmail.com gustavo.ferrazfontes@gmail.com
hewerton.souza@t-systems.com i.parmigiani@t-systems.com
igor.romulo.santos@gmail.com jeanderson.silva@t-systems.com
jl.oliveir@gmail.com jonathan.jimenez@iatec.com karinetrevisann@gmail.com
l.regioli@outlook.com leandro.piqueira@t-systems.com
lucas.p.oliveira@outlook.pt lxsilvareis@gmail.com
marcelohcordeiro@gmail.com marcio@almob.com.br
marcos.ammon@gmail.com moraesguga@gmail.com n.segeti@t-systems.com
nadson.dr11@gmail.com Paulo Scatena rafael.dias@t-systems.com
regis.maioli@t-systems.com reinaldo.rodrigues@t-systems.com
renan-silva@t-systems.com renan.baptista.cabral@gmail.com
rodrigo_7sch@hotmail.com tatidornel@gmail.com
thiago.fernandes@t-systems.com victor.prospero@t-systems.com
victor.santos@t-systems.com victor.souza@t-systems.com
vinicius.bonicio@t-systems.com webdeveloperrm@gmail.com
wesley.boccaletto@t-systems.com wsilva@t-systems.com
fabiano.felgas@gmail.com

Anexos  Integrações Assíncronas - Mensageria ○○○○○○

Resumo

Reunião focou em integrações assíncronas abordando conceitos de resiliência e a escolha entre orquestração ou coreografia.

Fundamentos de Integração Assíncrona

Debate sobre o uso de mensagens assíncronas enfatizou o desacoplamento de carga e a necessidade de considerar resiliência. Integrações assíncronas evitam latência sistêmica, mas exigem maior atenção aos componentes críticos.

Orquestração vs. Coreografia

Orquestração centraliza fluxos complexos facilitando ações compensatórias, enquanto coreografia favorece a descentralização em fluxos lineares. A decisão entre ambas depende de estabilidade e estrutura das equipes.

Garantia e Resiliência Técnica

Garantir a entrega exige padrões como Outbox e Inbox para assegurar idempotência e atomicidade. Estratégias de retry e Dead Letter Queue são essenciais para evitar perdas de mensagens.

Próximas etapas

- ☐ [O grupo] Disponibilizar Repositório: Tornar o repositório de código de exemplos disponível para acesso.
- ☐ [O grupo] Ler Artigo: Realizar leitura do artigo de Gregor Hope sobre o tratamento de erros em sistemas distribuídos.
- ☐ [O grupo] Revisar Implementações: Observar as dimensões de configuração de entrega ao decidir ou implementar um message broker. Comparar e analisar o código das implementações atuais.

- ☐ [Gabriel kohlrausch] Liberar Exemplos: Disponibilizar materiais e exemplos implementados no repositório Git para consulta.
- ☐ [Gabriel kohlrausch] Enviar Material: Compartilhar o material da reunião com os participantes. Disponibilizar os documentos em breve.
- ☐ [O grupo] Contatar Gabriel: Chamar o Gabriel no WhatsApp. Enviar mensagens em privado, conforme necessário.

Detalhes

- **Ajuste de Áudio e Check-in Inicial:** Gabriel Kohlrausch teve problemas com a qualidade do áudio no início da reunião, com a voz parecendo "metalizada" e distante. A situação foi resolvida após reconectar o fone, e Alex Silva Reis confirmou a melhoria. Em seguida, Gabriel iniciou a discussão perguntando sobre as ferramentas de mensageria e comunicação assíncrona que os participantes utilizam ([00:00:00](#)).
- **Flexibilidade Tecnológica e Preferências Pessoais:** Reinaldo Rodrigues compartilhou a visão de que, após anos de experiência em TI, é um "luxo" poder escolher o que gostar ou não, pois o trabalho exige versatilidade e a capacidade de superar adversidades, dadas as constantes mudanças em linguagens e ambientes ([00:01:40](#)).
- **Superando Preconceitos com Tecnologias:** Gabriel Kohlrausch concordou com Reinaldo, mas mencionou que tinha um preconceito significativo contra o Kafka até ter a oportunidade de trabalhar com ele em projetos reais, o que mudou drasticamente a sua visão. Ele observou que o Kafka resolve bons problemas e, para o desenvolvedor, a curva de aprendizado pode não ser tão íngreme quanto se espera, especialmente em comparação com ferramentas mais simples como RabbitMQ ou Azure Service Bus ([00:03:18](#)).
- **O Luxo de Gostar ou Não de Ferramentas:** Reinaldo Rodrigues reforçou que, em sua experiência anterior, atuando com várias companhias de seguro com diferentes filosofias e ferramentas de integração (como Oracle, Crystal Reports e mainframe), a necessidade era de "dar um jeito para fazer a coisa acontecer" ([00:04:53](#)). Ele concluiu que ter a preferência de gostar ou não gostar de uma ferramenta é um luxo para poucos ([00:06:11](#)).
- **Foco do Módulo: Integrações Assíncronas:** Gabriel Kohlrausch introduziu o foco principal da noite: o encerramento do módulo de integrações, abordando as integrações assíncronas ([00:06:11](#)). A intenção não é apenas ensinar o básico, mas aprofundar-se no dilema entre o assíncrono e o síncrono e

discutir a resiliência, um detalhe frequentemente esquecido ao optar por integrações assíncronas ([00:07:29](#)).

- **Estrutura do Módulo de Integrações:** Karine Trevisan questionou a contagem das aulas, e Gabriel Kohlrausch esclareceu a estrutura do módulo: houve aulas sobre banco de dados, integrações síncronas (HTTP, leitura de arquivo, etc.), e a aula atual encerraria com integrações assíncronas, focando em *message broker* e desacoplamento de sistemas ([00:08:40](#)).
- **As Três Forças em Integrações de Sistemas:** Gabriel Kohlrausch apresentou três eixos fundamentais que devem ser considerados ao decidir sobre qualquer integração entre sistemas: Comunicação (síncrona/assíncrona), Coordenação (orquestrada/coreografada) e Consistência (atômica/eventual). Ele enfatizou que, ao optar pela comunicação assíncrona, a consistência eventual é automaticamente abraçada, o que deve ser comunicado ao negócio ([00:09:42](#)).
- **Evitar o Modismo da Comunicação Assíncrona:** Gabriel desaconselhou a adoção da comunicação assíncrona, coreografada e eventual como regra de modernização de sistemas, classificando isso como um "modismo". Ele sugeriu que as três forças devem sempre ser consideradas. A discussão posterior se concentraria em porquês, vantagens e desvantagens do assíncrono, orquestração, coreografia e resiliência ([00:11:20](#)).
- **Impacto da Comunicação Síncrona na Disponibilidade e Latência:** O acoplamento síncrono reduz a disponibilidade geral do sistema, uma vez que a disponibilidade total é a multiplicação da disponibilidade de cada serviço envolvido (por exemplo, 99.9% x 99.9% x 99.9% resulta em 99.7%). Além disso, o tempo de resposta é a soma dos tempos de resposta de todos os serviços chamados, o que aumenta a latência ([00:14:24](#)).
- **Direcionamento sobre Quando NÃO Usar Integração Assíncrona:** Gabriel sugeriu olhar para a dimensão de "quando não usar" o assíncrono para evitar o *overengineering*. Ele desaconselhou o uso assíncrono para consultas simples que apenas retornam dados, operações que exigem atomicidade real, cenários com SLAs agressivos de latência (pois o *broker* adiciona milissegundos) ou em MVPs e sistemas pequenos onde a complexidade do *message broker* não se justifica ([00:15:59](#)).
- **Ganhos Chave da Decisão Assíncrona:** Ao justificar a adoção de integrações assíncronas, deve-se destacar o desacoplamento temporal (os sistemas não precisam estar online ao mesmo tempo) e o desacoplamento espacial (o produtor não precisa conhecer todos os consumidores das suas mensagens). O principal ganho, na opinião de Gabriel, é o desacoplamento de carga, que permite uma melhor escalabilidade, pois o consumidor pode processar

mensagens no seu próprio ritmo, independentemente do volume de entrada ([00:19:18](#)).

- **A Criticidade do Message Broker na Arquitetura:** Felipe Neves levantou uma preocupação sobre a falha do *broker* no fluxo assíncrono ([00:22:04](#)). Gabriel Kohlrausch respondeu que, em um sistema onde a arquitetura depende majoritariamente da integração assíncrona, o *broker* se torna um componente crítico ([00:23:25](#)). A infraestrutura deve tratar esse componente como crítico, muitas vezes exigindo soluções de alta disponibilidade, como *clusters* internos ou fornecedores de nuvem com SLAs altos (99.999%), para mitigar o risco de perda de mensagens e indisponibilidade ([00:24:41](#)). Wallace Silva compartilhou que em seu uso do RabbitMQ, caso não consiga salvar no *broker*, a informação é salva em banco de dados e enviada ao Rabbit assim que ele se restabelece ([00:26:05](#)).
- **Vocabulário Alinhado: Comando, Evento e Mensagem:** Gabriel recomendou alinhar o vocabulário das equipes para diferenciar Comando, Evento e Mensagem. Um ****Comando**** é imperativo e intencional (ex: *processar pagamento*), geralmente tem um destinatário conhecido, pode ser rejeitado e segue o padrão um para um (fila ou *queue*) ([00:27:12](#)). Um ****Evento**** deve ser sempre no passado (ex: *boleto pago*), não pode ser rejeitado (pois já aconteceu) e segue o padrão um para N (pub/sub), notificando o ecossistema ([00:33:48](#)). A ****Mensagem**** é o envelope, o contêiner (com *headers* e metadados) que carrega o *payload*, que é o Comando ou o Evento ([00:35:10](#)) ([00:40:06](#)).
- **Diferenciação e Uso de Comandos (um para um):** Felipe Neves demonstrou dificuldade em entender o uso assíncrono de Comandos, já que o destinatário é conhecido ([00:28:24](#)). Gabriel explicou que o uso de Comandos com filas (queue) serve para aumentar a resiliência e escala de processamento, como em cenários de ETL ou processamento em paralelo. Por exemplo, um serviço pode gerar um comando (ex: *inserir work item*) para que outro serviço (o sistema core) o processe em seu próprio ritmo, usando múltiplos *consumers* para paralelizar a operação ([00:31:11](#)).
- **Eventos (um para N) e Desacoplamento de Subscribers:** Wallace Silva expressou dificuldade com eventos, temendo que *consumers* que estejam fora do ar percam o processamento. Gabriel Kohlrausch esclareceu que o modelo *pub/sub* garante que o evento publicado no *broker* fica disponível para múltiplos *subscribers* ([00:37:47](#)). Sistemas como o Service Bus criam uma fila para cada *subscriber*, garantindo que mesmo que um sistema fique fora do ar por tempo prolongado, as mensagens se acumularão e serão processadas quando retornar, mantendo o paralelismo e a independência ([00:39:09](#)).

- **Coordenação: Orquestração vs. Coreografia:** A discussão avançou para a coordenação de jornadas mais complexas, onde múltiplos sistemas precisam executar ações sequenciais (ex: criar pedido, pagar, verificar estoque, envio) ([00:41:11](#)). ****Orquestração**** usa um componente central (o orquestrador) que conhece e comanda toda a sequência da jornada, sabendo o que fazer em caso de falhas. ****Coreografia**** usa o **message broker** como elemento central; os serviços publicam eventos e reagem a eventos de outros, sem que ninguém conheça o fluxo total ([00:43:35](#)).
- **Estratégias de Ação Compensatória:** Natan Barros destacou que na Orquestração é mais fácil implementar ações compensatórias (como estornar um pagamento em caso de falha de estoque), pois o orquestrador tem a visão completa do fluxo e decide as ações de recuperação ([00:46:49](#)). Na Coreografia, as compensações são mais complexas, pois cada serviço teria que escutar o evento de falha (ex: **estoque indisponível**) e iniciar sua própria ação compensatória (ex: **estornar pagamento**), o que aumenta a complexidade ([00:47:56](#)).
- **CrITÉrios para Escolha entre Orquestração e Coreografia:** Gabriel Kohlrausch apresentou critérios para guiar a decisão entre os dois modelos, enfatizando que não há uma resposta determinística. Os critérios incluem: ****Estabilidade do Fluxo**** (fluxo estável favorece Orquestração; fluxo com evolução frequente favorece Coreografia), ****Estrutura do Time**** (Time dono da jornada favorece Orquestração; Times distintos favorecem Coreografia), ****Visibilidade Necessária**** (Forte necessidade de compliance/auditoria favorece Orquestração), e ****Complexidade do Fluxo**** (Fluxos com muita ramificação favorecem Orquestração; Fluxos mais lineares Coreografia) ([00:50:38](#)) ([00:54:22](#)).
- **Regras de Negócio e o Orquestrador:** Felipe Neves confirmou que a Orquestração implica ter uma aplicação com regras de negócio ([00:55:37](#)). Gabriel Kohlrausch esclareceu que essa aplicação contém a ****regra do fluxo****, ou seja, o que fazer em caso de resultado X ou Y, mas não a regra específica do serviço (ex: a regra de pagamento deve permanecer no serviço de pagamento) ([00:56:57](#)). O orquestrador conhece o fluxo total e decide as ações, mantendo a autonomia dos serviços individuais ([00:58:04](#)).
- **Riscos da Coreografia Mal Implementada:** Bruno Joaquim complementou que, por ser mais complexa, a Coreografia tem um alto risco de ser mal implementada, o que pode levar a problemas graves no fluxo distribuído. Ele citou um caso de modernização onde um fluxo distribuído com Coreografia foi revisitado e alterado para uma abordagem com Orquestração para resolver problemas ([00:58:04](#)).

- **Resultados da Orquestração vs. Coreografia e Importância da Equipe:** A discussão sobre orquestração e coreografia destacou que a orquestração teve um desempenho superior, não inerentemente por ser melhor, mas pela sua implementação. A importância da equipe foi enfatizada, pois se houver apenas uma equipe, adotar um modelo de coreografia pode não ser o mais sensato, já que isso acrescenta complexidade desnecessária sem ganhos em troca ([00:59:15](#)).
- **Demonstração de Conceitos de Coreografia no Código:** O propósito da demonstração não era focar em todos os códigos, mas sim no conceito de coreografia e orquestração em nível de código ([01:00:01](#)). Para simplificar o exemplo, foi utilizada uma arquitetura de coreografia em uma única aplicação, embora isso não faça sentido na prática, e os participantes foram solicitados a abstrair a ideia de ser uma única aplicação ([01:01:02](#)).
- **Estrutura da Coreografia com Contextos Distintos:** A aplicação de exemplo utiliza dois contextos distintos, um financeiro e um de inscrições, que são independentes em termos de banco de dados e APIs, mas coexistem no mesmo projeto ([01:01:02](#)). No fluxo de adicionar uma nova inscrição, o `*handler*` processa a criação da inscrição sem chamar diretamente o módulo financeiro ([01:02:06](#)).
- **Mecanismo de Coreografia Através de Eventos de Domínio:** A coreografia é implementada usando uma biblioteca que sobrecarrega o ``save changes`` do Entity Framework, capturando eventos de domínio adicionados no agregado ([01:02:06](#)). Ao criar uma nova inscrição, um evento de domínio é adicionado a uma lista na classe abstrata, e a biblioteca se encarrega de publicar este evento após o commit ([01:03:38](#)).
- **Coreografia: Publicação e Consumo de Eventos de Forma Independente:** O módulo de inscrições não tem dependência do módulo financeiro, pois dispara um evento para o `*message broker*`. Na coreografia, o próximo serviço (neste exemplo, o financeiro) é quem deve saber como continuar a jornada ao receber o evento, lendo-o a partir de um tópico ([01:04:57](#)).
- **Gerenciamento de Tópicos e Coreografia no Financeiro:** Foi mencionado que é possível usar um único tópico para diversos tipos de eventos se o contexto permitir, embora tópicos separados possam ser necessários para maior vazão. O módulo financeiro, de forma coreografada, sabe o que fazer ao receber o evento de nova inscrição, processando o pagamento e continuando o fluxo ([01:06:23](#)).
- **Diferença entre Coreografia e Orquestração e Lógica de Compensação (Fallback):** Um participante questionou o papel do orquestrador no design da coreografia, e foi esclarecido que na coreografia não há um orquestrador

central que saiba quem emitiu o evento ([01:09:13](#)). A decisão sobre como realizar o **fallback** em múltiplos serviços que consomem o mesmo evento é de negócio, e quanto maior a complexidade de paralelismo e **fallback**, mais vantajoso pode ser ter um orquestrador ([01:10:31](#)).

- **Tomada de Decisão de Design e Evolução da Arquitetura:** Em projetos que evoluem e ganham complexidade, como requisitos de auditoria, pode ser necessário migrar de coreografia para orquestração de forma incremental, sendo essencial conversar com o negócio e desenhar o fluxo ([01:11:55](#)). As decisões de arquitetura devem ser tomadas no "último momento responsável", e é fundamental entender os **tradeoffs** e requisitos para evitar um **Frankenstein** na frente ([01:12:53](#)).
- **Orquestração como Solução para Fluxos Complexos e Propagação Excessiva de Eventos:** Foi levantada a importância de conceitos como **downstream** e **upstream** em cenários de eventos, e a complexidade crescente na propagação de eventos (**event-command-event**) ([01:15:34](#)). Propagar eventos em cascata para além de três passos é desaconselhado devido à complexidade do **fallback**, e a orquestração com Saga pode resolver problemas de fluxo estável e controle de falhas de forma mais visível do que a coreografia **in-memory** ([01:16:52](#)).
- **Vantagens Visuais da Orquestração de Workflow:** Trabalhar com muitos eventos pode se tornar complexo, e a orquestração resolve a falta de visibilidade ao centralizar o fluxo ([01:19:18](#)). O uso de um **workflow core** permite ver claramente os passos de uma jornada (ex: inscrição, pagamento, agendamento) e definir o que fazer em caso de falha, garantindo que o orquestrador sabe como lidar com todo o processo ([01:20:22](#)).
- **Tratamento de Ações Irreversíveis e Falhas em Sistemas Distribuídos:** Ações irreversíveis, como enviar um e-mail ou um pagamento, destacam a complexidade do **two-phase commit** em sistemas distribuídos ([01:23:33](#)). É crucial que as equipes internalizem que as falhas acontecerão e se concentrem na estratégia a ser aplicada (ignorar, tentar novamente ou compensar), e a leitura de um artigo de 2005 sobre tratamento de erros em sistemas distribuídos foi fortemente recomendada ([01:27:28](#)).
- **Sagas e Estratégias de Tratamento de Erro:** O padrão Saga, implementável com coreografia ou orquestração, exige que cada passo da jornada seja pensado com uma estratégia de erro: **write off** (ignorar), **retry** (retentar) ou compensação ([01:28:45](#)). Foi sugerido um template de design para documentar qual estratégia será usada para cada etapa e o porquê, ajudando na documentação e evitando esquecimentos futuros ([01:30:15](#)).

- **Ferramentas e Conceito de Orquestração no .NET:** Várias bibliotecas para implementar orquestração no .NET foram mencionadas, incluindo Hangfire, Mass Transit, Workflow Core, e até mesmo Event Flow. Soluções como Azure Durable Function e AWS Step Function também se encaixam no conceito, reforçando que o conceito é mais importante do que a tecnologia específica ([01:31:48](#)).
- **Garantia de Entrega e Resiliência em Brokers:** O tópico de garantia de entrega é fundamental para a resiliência em ambientes com **message brokers**. Envolve a confirmação (**acknology**) entre o produtor e o **broker**, e entre o consumidor e o **broker**, e é um ponto crucial para evitar perdas ou duplicações de mensagens ([01:33:08](#)).
- **Desafios da Comunicação Assíncrona e a Necessidade de Dominar Quatro Pontos:** Quatro cenários críticos foram apresentados, ilustrando falhas na comunicação que podem levar a duplicação ou perda de mensagens, como o produtor cair antes de receber o **ack** ou o consumidor cair após o processamento, mas antes de enviar o **ack**. O domínio desses quatro pontos é essencial para especialistas e **seniors**, pois a dor de cada falha impacta diretamente o negócio ([01:43:27](#)).
- ****Comportamento de *`acknology`* em Diferentes Message Brokers**:** O *`acknology`* pode ser uma perda silenciosa se for automático e o **broker** cair, resultando em dados apenas no **buffer** ([01:44:44](#)). O comportamento varia: o RabbitMQ exige configuração do cliente para *`ack`* explícito ou **fire and forget**; o Service Bus tem *`ack`* nativo por padrão; e o Kafka depende de configurações de *`ack`* para o cluster e réplicas ([01:46:26](#)).
- ****Tradeoff entre Vazão (**Throughput**) e Garantia de Entrega**:** Forçar um *`acknology`* explícito no produtor garante a entrega, mas diminui a vazão, pois é necessário esperar a resposta do **broker**. Caso a vazão seja um requisito de qualidade, pode-se abrir mão da garantia de entrega, o que leva a uma nova decisão e **tradeoff** ([01:47:51](#)).
- **Configurações de Consumo e Filas de Mensagens Mortas (Dead Letter):** No consumidor, o modo **receive and delete** no Service Bus aumenta o **throughput** de leitura, mas impede o reprocessamento em caso de falha. O **pick lock** é o padrão para o Service Bus, garantindo a entrega ([01:49:23](#)). Além disso, RabbitMQ e Service Bus oferecem **Dead Letter Queues** nativas, enquanto o Kafka exige implementação manual ([01:50:56](#)).
- ****O Desafio do *`autocommit`* no Kafka**:** O Kafka não trabalha com **commit** por mensagem no consumidor, mas sim por tempo (**autocommit true**). Esse *`autocommit`* pode causar reprocessamento ou perda de dados

se o consumidor cair dentro da janela de tempo, sendo uma dor de cabeça comum ([01:54:18](#)).

- **Modos de Entrega e Abrangência da Garantia:** As equipes devem considerar sempre os modos de entrega: **At most once** (no máximo uma vez, pode perder, mas não duplica) e **At least once** (pelo menos uma vez, não perde, mas pode duplicar). A garantia de entrega deve ser pensada para a jornada completa, do produtor ao consumidor, e não apenas para um lado ([01:57:33](#)).
- **Modo Exatamente Uma Vez (Exactly Once):** Gabriel Kohlrausch explicou que o modo "somente uma vez" para entrega de mensagens é o mais difícil de garantir, pois exige mais técnicas e recursos do que apenas uma configuração padrão. Para garantir esse modo, o **consumer** deve ser capaz de processar a mensagem atomicamente, garantindo que o **broker** recebeu a mensagem e que o **consumer** comitou todas as alterações no banco de dados. Uma implementação que se aproxima disso é o *`Exact Once Semantics`* no Kafka, mas só funciona em cenários onde o consumo e a produção ocorrem dentro da mesma transação, sem interação com sistemas externos ([01:58:57](#)).
- **Garantia de Entrega e Idempotência:** Na prática, se a garantia "exatamente uma vez" for crucial, deve-se garantir "pelo menos uma entrega" e implementar a idempotência no consumidor para evitar o processamento duplicado da mensagem. O efeito de "exatamente uma vez" só é alcançado com confirmação síncrona, estratégias de **retry**, e reconhecimento (**acknowledge**) do **consumer** com idempotência. Essas são as práticas que mais contribuem para a resiliência ([02:00:23](#)).
- **O Padrão Outbox e Atomicidade:** O padrão **Outbox** resolve o problema de garantir que um evento não seja perdido se o banco de dados for comitado, mas houver uma falha ao enviar a mensagem. O **Outbox** evita a necessidade de incluir todo o processo em uma única transação, aproveitando a transação do banco de dados para salvar o pedido e o **outbox** na mesma operação ([02:01:53](#)). Um serviço em segundo plano é responsável por pegar a mensagem do **Outbox** e publicá-la, removendo-a ou atualizando-a somente após receber o reconhecimento de entrega ([02:03:16](#)).
- **Problemas de Ordem com Outbox e Múltiplos Workers:** Um desafio no uso do **Outbox** é garantir a ordem das mensagens quando há múltiplos **workers** (serviços em segundo plano) processando o **Outbox**. Garantir a ordem com um único **worker** leva à perda de escalabilidade, pois a escala horizontal é limitada. Para ter escala horizontal e ordem garantida, pode-se usar estratégias de **hashing** ([02:04:57](#)).

- **Estratégia de Hashing Determinístico para Ordenação:** Uma estratégia é trabalhar com **hashes** determinísticos para associar mensagens de um mesmo agregado (como um pedido) a um **worker** específico. O **Outbox** deve persistir o ID do agregado e o **hash** do **worker** correspondente, usando algoritmos como MD5 ou FNV ([02:06:32](#)). Isso permite que **workers** paralelos puxem apenas as mensagens de seus **slots** designados, agrupem-nas pelo ID e as enviem na ordem de criação ([02:08:15](#)).
- ****Condições de Corrida (*Race Condition*) no Outbox**:** Ao usar múltiplos **workers** em paralelo no **Outbox** sem se preocupar com ordenação, é necessário gerenciar **race conditions** ([02:08:15](#)). Isso pode ser resolvido com estratégias de **lock**, como **lock** de linha no banco de dados ou um **lock** distribuído (por exemplo, com Redis) para garantir que apenas um **worker** processe um conjunto de mensagens por vez. Algumas bibliotecas, como Silverback e Wolverine, implementam soluções distribuídas para remover essa complexidade do código ([02:09:41](#)).
- **Idempotência do Consumer com Inbox:** Para lidar com o reprocessamento de mensagens duplicadas (idempotência no lado do **consumer**), a estratégia é usar uma tabela **Inbox**. O **consumer** registra as mensagens processadas no **Inbox** e, se uma duplicata for detectada (por exemplo, o pagamento de um pedido já processado), ela é ignorada, e o **acknowledge** é enviado para removê-la da fila ([02:10:59](#)). A principal diferença é que o **consumer** gerencia isso no seu próprio código, sem a necessidade de um **worker** separado ([02:12:16](#)).
- **Chave de Deduplicação para Idempotência:** A chave para a idempotência do **consumer** é a escolha de uma chave de deduplicação adequada. Não é recomendável usar o ID da mensagem do **broker**, pois cada reenvio gera um ID novo ([02:12:16](#)). Uma boa prática é criar uma chave de domínio (de negócio) que identifique a unicidade da operação, como "pagamento do pedido 42", em vez do ID de uma entidade ou do agregado ([02:13:29](#)).
- **Estratégia para Mensagens de "Veneno" (Poison Messages) e Dead Letter Queue:** Mensagens que causam erros persistentes (seja por falha de banco de dados, erro de negócio ou alteração de contrato) e que podem levar a um **leg** (acúmulo de mensagens) com **retries** infinitos, exigem uma estratégia de tratamento ([02:13:29](#)). A solução é a **Dead Letter Queue** (DLQ), que armazena essas mensagens após um número X de tentativas de **retry**, para que possam ser reprocessadas posteriormente. No Kafka, isso precisa ser implementado manualmente, enquanto em **brokers** como RabbitMQ ou Service Bus, é uma funcionalidade nativa ([02:14:59](#)).

- **O Conceito de Thundering Herd (Efeito Manada):** O efeito **Thundering Herd** ocorre quando, após uma falha (como a queda de um **consumer** ou de um serviço **downstream**), todos os **consumers** tentam realizar o **retry** simultaneamente, sincronizados ([02:16:14](#)). Essa carga repentina pode derrubar o banco de dados ou o sistema, mesmo após ele ter se recuperado. A solução para evitar esse problema é adicionar **delays** (**ditter**) randômicos entre as retentativas ([02:17:26](#)).
- **Estratégias de Retry (Tentativas):** Devem ser consideradas três estratégias de **retry**. A primeira é o **retry** imediato em memória, para falhas intermitentes e rápidas, com poucas tentativas (no máximo cinco) ([02:18:49](#)). A segunda é o **retry** com **delay**, que usa o **broker** para colocar a mensagem em uma fila de **retry** para ser reprocessada após um tempo maior, liberando a fila principal. A terceira é o **retry** agendado, usado em casos como **rate limits**, onde o sistema pode agendar o reprocessamento para o momento em que o **rate limit** terminar ([02:20:11](#)).
- **Recomendação de Bibliotecas para Mensageria Assíncrona:** Gabriel Kohlrausch recomendou as bibliotecas Wolverine e Silverback como principais referências para implementação de sistemas assíncronos ([02:21:46](#)). O conhecimento de aspectos como **Graceful Shutdown** é fundamental tanto para sistemas assíncronos quanto síncronos ([02:23:17](#)).
- **Agendamento de Mensagens no Azure Service Bus e Kafka:** Wallace Silva perguntou sobre agendamento de **retries**, e Gabriel Kohlrausch confirmou que o Azure Service Bus possui estratégias nativas de **schedule retry** e **schedule message** ([02:24:28](#)). No entanto, no Kafka, a estratégia de agendamento precisa ser implementada manualmente via **consumer** ([02:25:47](#)).
- **Compatibilidade de Conceitos com Google Pub/Sub:** Igor Romulo perguntou sobre a aplicação dos conceitos discutidos com o Google Pub/Sub. Gabriel Kohlrausch aconselhou que os conceitos de resiliência e arquitetura (como **timeout**, **retry**, etc.) são universais e devem ser verificados em relação ao **broker** específico (Google Pub/Sub, SNS/SQS) ([02:26:49](#)).
- **Timeouts de Entrega no Consumer:** Os **timeouts** no **consumer** são importantes para evitar que um **consumer** fique "preso" (**stuck**) em um **loop** infinito ou travado em uma mensagem, o que impediria o consumo de outras mensagens. O Kafka gerencia isso de forma mais granular com dois **timeouts** que controlam o **keep alive** (processo vivo) e se o processo está travado no consumo de uma mensagem ([02:28:03](#)).
- **Comportamento do Consumo de Mensagens (Pub/Sub e Fila):** Marcelo Henrique perguntou sobre o comportamento do Kafka quando um serviço

fica fora do ar e depois retorna. Gabriel Kohlrausch explicou que a mensagem fica na fila, e quando o **consumer** volta, ele começa a consumi-la, sendo este o comportamento padrão para todos os sistemas **pub/sub** ([02:31:12](#)). O Kafka usa o conceito de **Consumer Group** para diferenciar consumidores de um mesmo tópico, permitindo que vários grupos consumam o mesmo tópico (simulando **pub/sub**) ([02:32:46](#)).

- **Preferência por Bibliotecas no .NET:** Anderson Perim perguntou sobre as bibliotecas preferidas para .NET, e Gabriel Kohlrausch expressou preferência pelo Wolverine sobre o MassTransit para o RabbitMQ, considerando o primeiro mais enxuto ([02:34:05](#)). Para o Kafka, ele prefere o Silverback, que possui um design **Kafka first**, com pontos de extensão para políticas e comportamentos (**behaviors**) que facilitam a implementação de funcionalidades como **Circuit Breaker** para **consumers** ([02:35:45](#)) ([02:41:22](#)).
- **Implementação de Circuit Breaker para Consumers:** Gabriel Kohlrausch descreveu a implementação de um **Circuit Breaker** para **consumers** em Kafka usando Silverback ([02:36:28](#)). O **Circuit Breaker** impede que o **consumer** continue tentando processar mensagens se um serviço **downstream** (chamado via HTTP, por exemplo) estiver falhando consistentemente ([02:37:20](#)). Essa estratégia para de consumir o tópico temporariamente em vez de descartar as mensagens para uma DLQ, pois a falha é externa e conhecida ([02:38:33](#)).
- **Modelos de Consumers e Abstração de Código:** Gabriel Kohlrausch demonstrou que o Silverback permite que o código do **consumer** se concentre no domínio, sem inferências ou decorações da biblioteca, isolando a lógica de consumo do mecanismo de **pulling**. Isso é um ponto forte que evita a repetição de código (criação de múltiplos projetos de **consumer** que fazem a mesma coisa) ([02:44:33](#)) ([02:48:20](#)).
- **Fontes de Conhecimento sobre Padrões de Arquitetura:** Farley Souza perguntou sobre as fontes para aprender os conceitos de resiliência e arquitetura (como **race conditions**) ([02:51:32](#)). Gabriel Kohlrausch indicou que esses padrões surgem da leitura sobre arquitetura, resiliência e disponibilidade, sendo importantes para a transposição de conceitos para diferentes linguagens ([02:52:26](#)). Ele recomendou livros como **Enterprise Integration Patterns** e **Release It!** (de Michael T. Nygard) como leituras obrigatórias ([02:53:42](#)).
- **Preferência por Autofac (Injeção de Dependência):** Gabriel Kohlrausch mencionou que prefere o Autofac para injeção de dependência devido à facilidade de organização e separação de módulos ([02:50:42](#)). Ele não havia

feito *benchmarks*, mas outros participantes relataram achar a aplicação um pouco mais rápida com o Autofac ([02:51:32](#)).

- **Anúncio de Surpresa para Quarta-feira:** Gabriel Kohlrausch anunciou que haverá uma surpresa muito grande para os participantes na próxima quarta-feira e recomendou que se organizassem para estar presentes. Farley Souza confirmou que a referência era à quarta-feira seguinte. Gabriel Kohlrausch ressaltou que a surpresa ocorrerá, a não ser que haja um imprevisto muito grande, o qual eles tentam sempre evitar ([02:57:50](#)).
- **Disponibilização de Materiais e Contato Futuro:** Gabriel Kohlrausch informou que o material discutido será disponibilizado em breve para os participantes. Eles incentivaram os participantes a entrar em contato com eles pelo WhatsApp, cujo número está disponível no grupo, seja por mensagem privada ou em grupo, e se comprometeram a responder sempre que possível. Anderson Perim e Farley Souza agradeceram e se despediram, e Gabriel Kohlrausch desejou um bom descanso a todos ([02:57:50](#)).

Revise as anotações do Gemini para checar se estão corretas. [Confira dicas e saiba como o Gemini faz anotações](#)

*Como está a qualidade de **destas observações**? [Responda a uma breve pesquisa](#) para nos dar seu feedback, incluindo o quanto as observações foram úteis para o que você precisa.*